

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Y.Varun Kumar Reddy	Reg. No.:	192424230
Section / Batch:	2024	Date:	24-08-2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X.

The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
 - Without stemming
 - Porter stemming
 - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant. Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
 - Original words
 - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children"]

for w in words:
    print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
 - Regular plurals
 - Words ending in **-es**
 - Words ending in **-ies**
 - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy
 - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Question 1 – Morphological Analysis Error in Biomedical Dataset

Explanation:

1. **Preprocessing:** Clean the biomedical text by converting words to lowercase and removing unnecessary symbols.
2. **Porter Stemming:** Apply Porter Stemmer to words such as:
 - infection → infect
 - infected → infect
 - infectious → infecti
 - infect → infect
3. **Stemming Error:** infectious → infecti is an incorrect stem because infecti is not a meaningful English word.
4. **Inflectional vs Derivational:**
 - **Inflectional morphology** changes the form of a word but keeps its basic meaning, e.g., infect → infected.
 - **Derivational morphology** creates a related/new word, e.g., infect → infectious.
 - Therefore, they should be handled differently because removing derivational suffixes can destroy important meaning.

SAMPLE PROGRAM:

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", ps.stem(word))
```

SAMPLE OUTPUT:

```
infection -> infect
infectious -> infecti
infected -> infect
infect -> infect
```

Question 2 – Error Analysis of Finite-State Morphological Parser

Explanation:

1. Words such as happiest, unbelievable, running, reordering, smartphones, and unreadable contain **multiple prefixes or suffixes**.
2. The parser fails because it may be designed to recognize only **one affix at a time**.
3. For example:
 - unreadable → un + read + able
 - reordering → re + order + ing
 - smartphones → smartphone + s
4. The FST should be modified to allow transitions through **multiple prefixes and suffixes** instead of stopping after one.
5. After modification, test the parser again and calculate:

Accuracy = Correct analyses / Total words × 100

6. The modified FST has slightly more states/transitions, so its computational complexity increases, but it can still work efficiently because FSTs are generally fast.

SAMPLE PROGRAM:

```
import re
def parser(word):
    prefixes = ["un", "re"]
    suffixes = ["ing", "able", "est", "s"]
    prefix = ""
    suffix = ""
    for p in prefixes:
        if word.startswith(p):
            prefix = p
            word = word[len(p):]
            break
    for s in suffixes:
        if word.endswith(s):
            suffix = s
            word = word[:-len(s)]
            break
    return prefix, word, suffix
words = ["happiest", "unbelievable", "running",
        "reordering", "smartphones", "unreadable"]
for w in words:
    print(w, "->", parser(w))
```

SAMPLE OUTPUT:

```
happiest -> ('', 'happi', 'est')
unbelievable -> ('un', 'believ', 'able')
running -> ('', 'runn', 'ing')
reordering -> ('re', 'order', 'ing')
smartphones -> ('', 'smartphone', 's')
unreadable -> ('un', 'read', 'able')
```

Question 3 – Error Analysis of Stemming in News Classification

Explanation:

1. Stemming converts different forms of a word into a common stem.
2. Porter Stemmer may produce stems such as:
 - organization → organ
 - organizer → organ
 - organizing → organ
 - organized → organ
3. This can cause **loss of meaning** because organization, organizer, and organ have different meanings.
4. Compare three methods:
 - **Without stemming:** Keeps original words.
 - **Porter stemming:** Reduces words aggressively.
 - **Lemmatization:** Converts words to meaningful base forms.
5. Usually, **lemmatization gives better semantic information** because it produces valid words.
6. Compare classifier accuracy for all three methods. The method with higher accuracy and less semantic information loss is preferred.

SAMPLE PROGRAM:

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ["organization", "organizer",
         "organizing", "organized"]
print("Original -> Stem -> Lemma")
for word in words:
    stem = ps.stem(word)
    lemma = lemmatizer.lemmatize(word)
    print(word, "->", stem, "->", lemma)
```

SAMPLE OUTPUT:

```
Original -> Stem -> Lemma
organization -> organ -> organization
organizer -> organ -> organizer
organizing -> organ -> organizing
organized -> organ -> organized
```

Question 4 – Morphological Error Analysis in Information Retrieval

Explanation:

The words can be classified as:

Word	Type	Explanation
watches	Inflectional	Plural form of watch
watching	Inflectional	-ing form of watch
washed	Inflectional	Past form of wash
washable	Derivational	Creates a new related word
washer	Derivational	Creates a new word for a person/device

1. Porter Stemmer may remove suffixes too aggressively.
2. **Inflectional suffixes** can usually be removed because the basic meaning remains similar.
3. **Derivational suffixes** should often be preserved because they can change the meaning or part of speech.
4. A better strategy is to use **lemmatization** or a morphology-aware system instead of blindly removing suffixes.
5. This improves product-search relevance because words with different meanings are not incorrectly treated as the same word.

SAMPLE PROGRAM:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["watches", "watching", "washable",
        "washer", "washed"]
for word in words:
    print(word, "->", ps.stem(word))
```

SAMPLE OUTPUT:

```
watches -> watch
watching -> watch
washable -> washabl
washer -> washer
washed -> wash
```


Question 5 – Identify the Error, Rectify It, and Analyze the Output

Explanation:

The main error is:

ps.stem is designed for **one word**, but data["Text"] contains an entire sentence/document.

For example:

"The organization is organizing an event"

should not be passed directly to ps.stem().

Instead:

1. Read the BBC News dataset.
2. Take each news article.
3. Split the text into individual words.
4. Apply Porter Stemmer to each word.
5. Join the stemmed words back together.
6. Compare the original and stemmed words.

Example:

- running → run
- studies → studi
- organization → organ
- organized → organ
- happiness → happi

Some stems such as studi, organ, and happi are not actual English words. These are examples of **over-stemming**.

For the 20 cases, identify whether the removed part represents:

- **Inflectional morphology:** running → run
- **Derivational morphology:** organization → organ

The main problem is that Porter Stemmer focuses on reducing words rather than preserving their exact meaning.

SAMPLE PROGRAM:

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# Sample BBC-style news articles
data = [
    "The organization is organizing a new event",
    "The organizer organized the meeting",
    "Researchers are studying new technology",
    "The company is developing new products",
    "The government announced new policies"
]

# Function to stem complete sentences
def stem_text(text):
    words = text.split()
    return " ".join(ps.stem(word) for word in words)

# Compare original and stemmed articles
print("ARTICLE ANALYSIS\n")

for text in data:
    print("Original :", text)
```

```
print("Stemmed :", stem_text(text))
print()
```

```
# Analyze 20 words
```

```
words = [
    "organization", "organizer", "organizing", "organized",
    "running", "runner", "studies", "studying", "studied",
    "happiness", "beautiful", "easily", "relational",
    "connection", "traditional", "national", "universities",
    "computers", "boxes", "technology"
]
```

```
print("20 STEMMING CASES\n")
print("Original -> Stemmed")
```

```
for word in words:
    print(word, "->", ps.stem(word))
```

SAMPLE OUTPUT:

```
ARTICLE ANALYSIS

Original : The organization is organizing a new event
Stemmed  : the organ is organ a new event

Original : The organizer organized the meeting
Stemmed  : the organ organ the meet

Original : Researchers are studying new technology
Stemmed  : research are studi new technolog

Original : The company is developing new products
Stemmed  : the compani is develop new product

Original : The government announced new policies
Stemmed  : the govern announc new polici

20 STEMMING CASES

Original -> Stemmed
organization -> organ
organizer -> organ
organizing -> organ
organized -> organ
running -> run
runner -> runner
studies -> studi
studying -> studi
studied -> studi
happiness -> happi
beautiful -> beauti
easily -> easili
relational -> relat
connection -> connect
traditional -> tradit
national -> nation
universities -> univers
computers -> comput
```

Question 6 – Error Analysis of Finite-State Morphological Parser

Explanation:

The original parser has a major logical error:

`word[: -2]`

removes **two characters**, even when only `s` should be removed.

For example:

`cars` → `car` should happen, but the program removes `rs`.

Different plural rules must be handled separately:

- `car` → `cars`
- `box` → `boxes`
- `city` → `cities`
- `child` → `children`

A corrected parser should check:

1. **Irregular plurals:** `children` → `child`
2. **-ies:** `cities` → `city`
3. **-es:** `boxes` → `box`
4. **Regular -s:** `cars` → `car`

The parser can be tested using WordNet or another English vocabulary.

Limitations of rule-based FST:

- It cannot easily handle all English exceptions.
- Irregular words require special rules.
- New/slang words may not be recognized.
- Multiple morphological patterns make the rules more complex.

So, FSTs are **fast and useful for regular patterns**, but they need additional lexical information to handle irregular English words.

SAMPLE PROGRAM:

```
def parser(word):
```

```
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse"
    }
```

```
    if word in irregular:
        return irregular[word], "Plural"
```

```
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural"
```

```
    if word.endswith("es"):
        return word[:-2], "Plural"
```

```
    if word.endswith("s"):
        return word[:-1], "Plural"
```

```
    return word, "Singular"
```

```
words = ["cars", "boxes", "cities", "children",
        "men", "books"]
```

```
for w in words:  
    print(w, "->", parser(w))
```

SAMPLE OUTPUT:

```
cars -> ('car', 'Plural')  
boxes -> ('box', 'Plural')  
cities -> ('city', 'Plural')  
children -> ('child', 'Plural')  
men -> ('man', 'Plural')  
books -> ('book', 'Plural')
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____